

מדריך ליצירת מערכת Posts

מדריך זה מנחה אותך כיצד ליישם מערכת Posts מלאה עם אימות, העלאת קבצים ל-S3 ו-`validations`.

סקירה כללית

המשימה כוללת:

- יצירת Model חדש ל-Post
- יצירת Controller ל-Post
- יצירת Routes ל-Post
- הוספת אימות לכל ה-routes
- טיפול בהעלאת תמונות ל-S3 בעת יצירת פוסט
- הוספת `validations` ליצירת פוסט (בנוסף)

שלב 1: יצירת Post Model

1.1 יצירת הקובץ

צור קובץ חדש: `models/postModel.js`

1.2 מבנה ה-Schema

השתמש ב-`userModel.js` כדוגמה. ה-Post Model צריך לכלול:

```
import mongoose from "mongoose"

const postSchema = new mongoose.Schema({
  uploaded_by: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true
  },
  text: {
    type: String,
    required: true
  },
  image: {
    type: String,
    default: ""
  },
  likes: {
    type: Number,
    default: 0
  }
}, {
  timestamps: true // אוטומטית created_at ו updated_at יוסיף
})
```

```
export default mongoose.model("Post", postSchema)
```

הערות:

- `uploaded_by` הוא reference ל-ObjectId (User model)
- `likes` מתחיל ב-0 כברירת מחדל
- `image` יכול להיות ריק אם לא הועלה קובץ

שלב 2: יצירת Post Controller

2.1 יצירת הקובץ

צור קובץ חדש: `controllers/postController.js`

2.2 יצירת הפונקציות

השתמש ב-`userController.js` ו-`uploadController.js` כדוגמאות.

2.2.1 פונקציית יצירת פוסט (`createPost`)

```
async createPost(req, res) {
  try {
    // 1. אם כן, העלה ל-S3 (req.file) בדוק אם יש קובץ
    // 2. URL-קבל את ה-S3
    // 3. עם הנתונים post-צור את ה:
    //    - uploaded_by: req.user._id (של האימות middleware-מה)
    //    - text: req.body.text
    //    - image: URL-מה ה-S3 (או "" אם יש קובץ)
    //    - likes: 0
    // 4. החזר תשובה 201 עם הפוסט שנוצר
  } catch (error) {
    // טיפול בשגיאות
  }
}
```

טיפים:

- השתמש ב-`uploadFileToS3` מ-`services/s3Service.js` (כמו ב-`uploadController.js`)
- השתמש ב-`postModel.create()` ליצירת הפוסט
- אם יש `req.file`, העלה אותו ל-S3 וקבל את ה-URL
- אם אין קובץ, השאר את `image` כ-string ריק

2.2.2 פונקציית קבלת כל הפוסטים (`getAllPosts`)

```
async getAllPosts(req, res) {
  try {
```

```
// 1. כדי לקבל את כל הפוסטים postModel.find()
// 2. כדי לקבל את פרטי המשתמש .populate('uploaded_by')
// 3. החזר תשובה 200 עם כל הפוסטים
} catch (error) {
  // טיפול בשגיאות
}
}
```

טיפים:

- השתמש ב-`populate('uploaded_by', 'name email')` כדי לקבל רק שדות מסוימים מהמשתמש
- אפשר למיין לפי תאריך: `-1. createdAt` (`{ sort: { createdAt: -1}`) (החדשים ראשונים)

2.2.3 פונקציית מחיקת פוסט (`deletePost`)

```
async deletePost(req, res) {
  try {
    // 1. id-קבל את ה query: req.query.id
    // 2. id מצא את הפוסט לפי id
    // 3. בדוק שהמשתמש הוא זה שיצר את הפוסט (uploaded_by === req.user._id)
    // 4. (Forbidden) אם לא - החזר שגיאה 403
    // 5. אם כן - מחק את הפוסט
    // 6. החזר תשובה 200 עם הודעת הצלחה
  } catch (error) {
    // טיפול בשגיאות
  }
}
```

טיפים:

- השתמש ב-`postModel.findByIdAndDelete()` או `postModel.findById().deleteOne()`
- השווה בין `post.uploaded_by.toString()` ל-`req.user._id.toString()` (כי הם `ObjectId`)

2.3 ייצוא ה-Controller

```
const postCtrl = {
  createPost,
  getAllPosts,
  deletePost
}

export default postCtrl
```

שלב 3: יצירת Post Routes

3.1 יצירת הקובץ

צור קובץ חדש: `routes/postRoutes.js`

3.2 הגדרת ה-Routes

השתמש ב-`userRoutes.js` ו-`uploadRoutes.js` כדוגמאות.

```
import express from "express"
import postCtrl from "../controllers/postController.js"
import authUser from "../middlewares/auth.js"
import { uploadSingle } from "../middlewares/upload.js"
// validations: אם תוסיף
// import { createPostValidation } from "../validation/postValidation.js"

const router = express.Router()

// POST /api/post - יצירת פוסט חדש
// (בנוסף), validation (אם יש קובץ), authUser, uploadSingle: צריך
router.post("/", authUser, uploadSingle, postCtrl.createPost)

// GET /api/post - קבלת כל הפוסטים
// צריך: authUser
router.get("/", authUser, postCtrl.getAllPosts)

// DELETE /api/post?id=... - מחיקת פוסט
// צריך: authUser
router.delete("/", authUser, postCtrl.deletePost)

export default router
```

הערות:

- כל ה-routes עוברים דרך `authUser` middleware
- ה-route POST משתמש גם ב-`uploadSingle` כדי לקבל את הקובץ
- הסדר חשוב: `authUser` לפני `uploadSingle`

3.3 הוספת ה-Routes ל-mainRoutes

ערוך את `routes/mainRoutes.js`:

```
import postRoutes from "./postRoutes.js"

// הוסף את השורה הבאה:
router.use("/post", postRoutes)
```

שלב 4: טיפול ב-FormData ו-S3 Upload

4.1 במידה ויש קובץ

בפונקציית `createPost` ב-controller:

```
let imageUrl = ""

if (req.file) {
  const fileBuffer = req.file.buffer
  const originalName = req.file.originalname
  const mimeType = req.file.mimetype

  const { s3Url } = await uploadFileToS3(fileBuffer, originalName, mimeType)
  imageUrl = s3Url
}
```

4.2 יצירת הפוסט

```
const post = await postModel.create({
  uploaded_by: req.user._id,
  text: req.body.text,
  image: imageUrl,
  likes: 0
})
```

הערות:

- `req.body.text` יגיע מה-`formData`
- `req.file` יגיע מה-`uploadSingle` middleware
- אם אין קובץ, `imageUrl` יישאר כ-`string` ריק

שלב 5: הוספת Validations (בנוס)

5.1 יצירת קובץ Validation

צור קובץ חדש: `validation/postValidation.js`

5.2 יצירת Schema

השתמש ב-`userValidation.js` כדוגמה:

```
import Joi from "joi"
import { createValidation } from "../middlewares/validation.js"

const createPostValidation = createValidation(
  Joi.object({
    text: Joi.string().required().min(1).max(500).messages({
      "string.empty": "Text is required",
      "string.min": "Text must be at least 1 character",
      "string.max": "Text must not exceed 500 characters"
    })
  })
)
```

```

    })
  })
)

export { createPostValidation }

```

הערות:

- ה-validation בודק רק את **text** כי הקובץ נשלח ב-**req.file** ולא ב-**req.body**
- אם תרצה, תוכל להוסיף validation גם על הקובץ (בדיקת סוג קובץ, גודל וכו')

5.3 הוספת Validation ל-Route

ערוך את **routes/postRoutes.js**:

```

import { createPostValidation } from "../validation/postValidation.js"

router.post("/", authUser, uploadSingle, createPostValidation,
  postCtrl.createPost)

```

הערות:

- הסדר חשוב: **authUser** → **uploadSingle** → **createPostValidation** → **postCtrl.createPost**
- ה-validation יבדוק את **req.body.text** אחרי שהקובץ כבר עבר דרך **uploadSingle**

שלב 6: בדיקה

6.1 בדיקת יצירת פוסט

```

POST http://localhost:YOUR_PORT/api/post
Authorization: YOUR_TOKEN
Content-Type: multipart/form-data

text: "זה הפוסט הראשון שלי"
file: [בחר קובץ תמונה]

```

6.2 בדיקת קבלת כל הפוסטים

```

GET http://localhost:YOUR_PORT/api/post
Authorization: YOUR_TOKEN

```

6.3 בדיקת מחיקת פוסט

```
DELETE http://localhost:YOUR_PORT/api/post?id=POST_ID
Authorization: YOUR_TOKEN
```

סיכום הקבצים ליצירה/עריכה

קבצים חדשים ליצירה:

- 1. ☒ `models/postModel.js`
- 2. ☒ `controllers/postController.js`
- 3. ☒ `routes/postRoutes.js`
- 4. ☒ `validation/postValidation.js` (בנוסף)

קבצים לעריכה:

- 1. ☒ `routes/mainRoutes.js` - הוספת `postRoutes`, `router.use("/post",`

טיפים נוספים

1. **טיפול בשגיאות:** ודא שכל הפונקציות ב-controller מטפלות בשגיאות כראוי
2. **בדיקת הרשאות:** במחיקה, ודא שרק המשתמש שיצר את הפוסט יכול למחוק אותו
3. **Populate:** השתמש ב-`populate()` כדי לקבל פרטים על המשתמש שיצר את הפוסט
4. **Sorting:** אפשר למיין את הפוסטים לפי תאריך (החדשים ראשונים)
5. **Optional Image:** הפוסט יכול להיות ללא תמונה - ודא שהקוד מטפל בזה

בעיות נפוצות

הקובץ לא עולה ל-S3

- ודא שה-`env` מכיל את כל המשתנים הנדרשים ל-AWS
- בדוק שה-`uploadSingle` middleware מופיע לפני ה-controller

Validation לא עובד

- ודא שה-`createPostValidation` מופיע אחרי `uploadSingle`
- בדוק שה-`text` נשלח ב-`req.body` ולא ב-`req.file`

מחיקה לא עובדת

- ודא שאתה משווה בין `ObjectId` בצורה נכונה (`().toString()`)
- בדוק שה-`id` נשלח ב-`req.query.id` ולא ב-`req.params.id`

 **בהצלחה!**